Relatório Final de Iniciação Científica

Discente: Gustavo Ferreira Leite de Barros (20240020300) **Orientador:** Alexandre Cabral Mota **Centro:** Centro de Informática (CIn) — UFPE **Projeto:** PIC2192-2025 — Verificação Formal e Sugestão Automática de Propriedades em Contratos Inteligentes: Integrando LLMs e Preservação Semântica **Plano de Trabalho:** Abordagem Experimental para Comprovação de Otimização de Gás em Contratos Inteligentes Solidity **Período de execução:** 01/09/2025 a 31/08/2026 **Situação atual do projeto:** ( ) Em andamento ( ) Concluído *(confirmar antes do envio)*

---

1. Introdução

Contratos inteligentes são programas executados em blockchain que automatizam transações e serviços digitais sem intermediário central. Como administram diretamente recursos financeiros e, uma vez implantados, são difíceis ou impossíveis de modificar, uma vulnerabilidade ou uma alteração incorreta no código pode causar prejuízo financeiro imediato e irreversível — ao contrário de software convencional, não há como aplicar um patch de emergência depois que o contrato já está em produção na rede.

Um segundo fator de pressão sobre esse código é o custo de execução: toda operação em uma rede como a Ethereum consome **gás**, cobrado do usuário em cada transação. Esse custo cria um incentivo real para otimizar contratos — reduzir armazenamento, simplificar laços, eliminar checagens redundantes. O problema é que essas mesmas otimizações são uma fonte comum de regressão de segurança: uma checagem removida por parecer redundante pode ser exatamente a checagem que impedia uma exploração. Otimizar sem verificar é trocar custo de gás por risco de exploração.

Este trabalho parte dessa tensão para propor e validar uma abordagem automatizada que combina **análise estática** (Slither), **modelos de linguagem** (LLM) e **verificação formal** (Certora Prover) em uma pipeline única: identificar vulnerabilidades reais, formalizar as propriedades de segurança violadas, corrigir o contrato e **provar matematicamente** — não apenas testar por exemplos — que a versão corrigida preserva o comportamento esperado. É essa capacidade de prova formal que torna a abordagem adequada como rede de segurança para mudanças no contrato, incluindo mudanças motivadas por otimização de gás: qualquer alteração no código, por economia ou por correção, pode ser revalidada contra as mesmas propriedades formais antes de ser aceita.

2. Objetivos

2.1 Objetivo geral

Projetar, implementar e validar uma pipeline automatizada que identifique, diagnostique, corrija e valide formalmente vulnerabilidades de segurança em contratos inteligentes escritos em Solidity, combinando análise estática, modelos de linguagem e verificação formal, de modo que o mesmo mecanismo sirva como salvaguarda para mudanças no contrato motivadas por otimização de custo de execução (gás).

2.2 Objetivos específicos

- Integrar análise estática (Slither) como fonte determinística de evidência de vulnerabilidade.
- Usar um LLM para planejar quais achados são formalizáveis, gerar as propriedades em CVL (Certora Verification Language) e diagnosticar/corrigir os casos confirmados.
- Usar o Certora Prover para confirmar formalmente violações de propriedade e obter contraexemplos concretos, e para revalidar o contrato corrigido.
- Construir salvaguardas determinísticas (*guardrails*) que limitem o que o LLM pode gerar ou alterar, reduzindo o risco de correções incorretas ou fora de escopo.
- Avaliar a abordagem em um benchmark controlado e em contratos reais de auditorias públicas (Code4rena), medindo cobertura, taxa de resolução e minimalidade das correções.
- Investigar a generalização da pipeline para além dos padrões cadastrados manualmente, incorporando técnicas recentes da literatura (prompts de perspectiva, contexto por grafo de chamadas, geração de casos de teste por mutação).

3. Fundamentação Teórica

A abordagem determinística por detector — mapear cada tipo de achado do Slither a uma regra de diagnóstico e correção escrita à mão — tem limite estrutural conhecido: cada classe nova de vulnerabilidade exige código novo, e a cobertura fica presa ao que foi cadastrado manualmente. Para endereçar essa limitação, o trabalho se apoiou em três artigos recentes (2025–2026) da ACM Transactions on Software Engineering and Methodology (TOSEM), cada um cobrindo um ângulo diferente do problema de detecção de vulnerabilidades em contratos inteligentes com LLMs:

CodeXplain (Chen et al.) propõe gerar, para cada contrato, explicações de código guiadas por nove *prompts de perspectiva* fixos (ex.: análise de fluxo lógico, controle de acesso, gestão de estado), que alimentam um classificador treinado. Relata F1 de 94,12% em 3.544 contratos reais. A contribuição aproveitada aqui foi a ideia dos prompts de perspectiva como substituto de um dicionário fixo de dicas de correção por detector.

MANDO-LLM (Nguyen et al.) representa o contrato como grafo heterogêneo construído a partir de um AST obtido via **tree-sitter** — não via compilador —, o que permite analisar código que não compila, situação comum em repositórios reais grandes. Relata melhorias de F1 de até 80,72% no

nível de contrato e detecção no nível de linha. A contribuição aproveitada foi a adoção de tree-sitter como camada de parsing independente de compilação.

CCIHunter (Li et al.) detecta inconsistências entre o código implementado e a intenção declarada em comentários (NatSpec), usando profundidade de expansão de chamada de função limitada a 2 níveis — limite validado empiricamente pelos autores em 14.039 funções reais (84,50% de profundidade 1, 11,97% de profundidade 2) — e usa sete operadores de mutação de código (troca de operador aritmético/binário/unário, remoção de evento, remoção de modifier, troca de `payable`, troca de visibilidade de função) como técnica de pré-treino sem rótulo. Relata precisão de 0,95 e F1 de 0,93. As contribuições aproveitadas foram o contexto de chamada em profundidade 2 e os operadores de mutação como gerador de casos de teste de regressão.

4. Metodologia

4.1 Arquitetura da pipeline

A pipeline é organizada em seis etapas sequenciais:

1. **Análise estática.** O Slither analisa o contrato-fonte e os achados brutos são normalizados (`agent/core/slither_normalizer.py`) em classes estáveis de vulnerabilidade, desacoplando o restante da pipeline do formato de saída específico da ferramenta.
2. **Planejamento formal.** Um LLM decide, entre os achados normalizados, quais são candidatos a verificação formal, com base em um catálogo de vulnerabilidades (`vulnerability_catalog.py`) que classifica cada classe como formalizável via Certora, estática-confirmada ou apenas revisão manual.
3. **Geração de propriedade formal.** Para cada candidato, é gerada uma especificação em CVL. Um validador determinístico (`tools/spec_validator.py`) corrige automaticamente erros sintáticos comuns do CVL gerado pelo LLM (blocos `methods{}` mal formados, tipos incompatíveis, instruções múltiplas em uma linha, regras planejadas mas nunca escritas), sem gastar uma chamada de LLM adicional para isso.
4. **Verificação formal.** O Certora Prover roda a especificação contra o contrato original. Uma violação confirmada vem acompanhada de um contraexemplo concreto (valores de entrada que quebram a propriedade), extraído e reaproveitado nas etapas seguintes.
5. **Diagnóstico e correção.** Para classes de vulnerabilidade conhecidas, o diagnóstico é determinístico (sem chamada de LLM); para o restante, o LLM diagnostica a causa raiz e propõe um patch, recebendo o contrato completo para preservar imports, structs, modifiers e invariantes locais.
6. **Validação do patch.** Um *patch guard* (`core/patch_guard.py`) verifica que a correção é mínima — bloqueia mudança de assinatura pública, remoção de variável de estado, alteração de string literal fora de escopo — antes de o Slither e o Certora rodarem novamente sobre a

versão corrigida. Se a primeira tentativa é bloqueada, uma nova tentativa (até 3 no total) recebe o relatório do guard e o contraexemplo da falha anterior.

Essa combinação de LLM com validadores determinísticos foi uma escolha deliberada de projeto: usar o LLM onde julgamento é necessário (planejamento, diagnóstico semântico, geração inicial de código/CVL) e reservar regras determinísticas para tudo que é decidível sintaticamente — o que reduz custo de chamadas ao modelo e elimina uma classe inteira de erro (a LLM "esquecer" de escrever uma regra que ela mesma planejou, por exemplo).

4.2 Generalização além do catálogo manual

A segunda frente de trabalho, motivada pela literatura descrita na Seção 3, endereçou a limitação estrutural de depender de um catálogo fixo de detectores:

- **Prompts de perspectiva** (`core/perspectives.py`): implementadas as nove perspectivas do CodeXplain e a tabela de mapeamento vulnerabilidade → subconjunto de perspectivas, injetadas no prompt de diagnóstico sem custo de chamada extra de API.
- **Contexto por grafo de chamadas** (`core/call_expansion.py`): busca em largura no grafo de chamadas a partir da função do achado, profundidade 2 (valor validado empiricamente pelo CCIHunter), incluindo modifiers aplicados como chamada — é onde normalmente mora o guard de controle de acesso. Isso corrigiu um caso real de diagnóstico falso: um `require` de autorização dentro de um modifier sem parênteses não aparecia no contexto enviado ao LLM, que diagnosticava "controle de acesso ausente" em uma função que na verdade já era protegida.
- **Parsing independente de compilador** (`core/treesitter_source.py`): adoção de tree-sitter como alternativa ao parser por expressão regular, com fallback automático. Validado em 2.035 arquivos `.sol` reais.
- **Operadores de mutação como suíte de regressão** (`core/mutation.py`): os sete operadores do CCIHunter aplicados na direção inversa — a partir de um contrato já corrigido, reinjetam um defeito por vez — para gerar casos de teste sem depender de um dataset rotulado externo.

4.3 Avaliação

A avaliação foi feita em duas frentes: um benchmark controlado de contratos escritos especificamente para exercitar classes conhecidas de vulnerabilidade, e um conjunto de contratos reais extraídos de auditorias públicas do Code4rena, usados como teste de generalização sem nenhum ajuste específico de regra para o contrato-alvo.

5. Resultados

5.1 Benchmark controlado

Contrato	Achados Slither	Selecionados	Confirmados	Resolvidos	Taxa
SimpleBank	12	4	4	4	4/4
King	9	4	4	4	4/4
CrowdfundingVault	16	4	4	4	4/4
DeFiVault	19	5	5	5	5/5
ERC2771MulticallVulnerable	8	1	1	1	1/1
EnterpriseTreasury300 (327 linhas)	26	9	9	9	9/9

Três contratos adicionais (`TimelockVault`, `AnotherVulnerableBank`, `VulnerableBankToken`) serviram de controle negativo — a pipeline corretamente não gerou nenhum candidato acionável, evitando falso positivo.

5.2 Contratos reais (Code4rena)

Projeto / arquivo	Tamanho	Resultado
Solmate `WETH.sol`	—	0 candidatos acionáveis (controle negativo real)
Forgotten Runes `ForgottenRunesWarriorsMinter.sol`	—	71 achados normalizados → 2/2 resolvidos (`missing-zero-check`)
Inverse `Market.sol`	621 linhas	63 achados normalizados → 3/3 resolvidos (`missing-zero-check`)
Escher `FixedPrice.sol`	—	0 candidatos — evitou falso positivo em `selfdestruct` interno
Caviar `PrivatePool.sol`	794 linhas	0 candidatos acionáveis; validou compilação de repositório Foundry real com `remappings.txt`
Astaria (`PublicVault.sol`, `AstariaRouter.sol`, `LienToken.sol`)	725–919 linhas	Bloqueado — dependência de submódulo privado não pôde ser clonada no ambiente de teste

Em todos os casos resolvidos, o *patch guard* aprovou a correção final com 0 erros e 0 avisos fora de escopo, e a revalidação (Slither + Certora) confirmou a eliminação da vulnerabilidade sem regressão nas demais propriedades.

5.3 Cobertura do catálogo em código real

Uma medição feita sobre 5 contratos reais do projeto Inverse (108 achados normalizados de Slither) quantificou a limitação estrutural que motivou o trabalho de generalização:

Categoria	Achados	%
Formalizável (chega ao Certora)	18	17%
Catalogado, mas não formalizável	22	20%
Fora do catálogo	68	63%

Dos 18 achados formalizáveis, 17 são `missing-zero-check` e 1 é `tx-origin`. Entre os 68 fora do catálogo há tanto ruído informacional (convenção de nomes, versão de solc) quanto vulnerabilidade de segurança real ainda não coberta por uma regra formal — `reentrancy-events` (11 ocorrências), `unchecked-transfer` (7), `divide-before-multiply` (5), `incorrect-equality` (2). Esse número — 17% de cobertura formal em código real — é o argumento quantificado que justifica investir em generalização (Seção 4.2) em vez de continuar apenas adicionando classes manualmente ao catálogo.

5.4 Cobertura total do catálogo e execução sem LLM (novembro/2026)

O catálogo de vulnerabilidades passou a cobrir os 99 detectores do Slither 0.11.3 (105 entradas); um teste automatizado falha se uma versão nova do Slither listar um detector desconhecido. Foi criado o modo `--static-only`, que executa Slither, catálogo, patches determinísticos e reanálise sem nenhuma chamada a LLM, e um detector próprio (`unrestricted-write`).

Sobre 201 dos 264 arquivos dos projetos reais, o modo sem LLM confirmou 77 correções pela reanálise do Slither e deixou 16 achados críticos sem correção. Os 63 arquivos restantes não puderam ser analisados por limites de ambiente (dependência privada indisponível, *stack too deep*, testes de `forge-std`).

Com a pipeline completa nos 10 contratos do registro, 7 achados acionáveis foram aprovados e 28 foram resolvidos: 21 por patch determinístico e 7 pelo LLM, todos verificados pelo Certora. Cada contrato foi executado uma vez, com modelo diferente do usado nos experimentos originais.

No conjunto DeFiVulnLabs (57 arquivos), 53 foram analisados, com 702 achados em 42 tipos de detector; 29 arquivos foram corrigidos sem LLM. Nos 26 arquivos com achados críticos, a execução com LLM e Certora aprovou 9. Uma auditoria dessas execuções mostrou que 12 de 19 reparos recaíam em contratos de teste (`ContractTest`, `Attack`) e não no contrato vulnerável; passou-se a descartar esses contratos, escolhendo para verificação o contrato com mais achados críticos. Uma reexecução posterior, com essas correções, aprovou 4 arquivos (`phantom-permit`, `txorigin`, `Storage-collision`, `Storage-collision-audio`). Dois não foram aprovados: `Randomness`, cujo achado `arbitrary-send-eth` não tem regra formal por decisão de projeto, e `fee-on-transfer`, cujo patch do LLM foi rejeitado pelo *patch guard* por alterar um literal de string.

Ressalva: a cota diária do provedor de LLM se esgotou no modelo principal, e parte da rodada usou um modelo menor, com desempenho inferior; os números do LLM não são comparáveis diretamente aos da Seção 5.1.

6. Discussão e limitações

Os resultados no benchmark controlado (33/33 correções resolvidas nas classes cobertas) mostram que, quando a vulnerabilidade pertence a uma classe modelada, a pipeline resolve com alta confiabilidade e correção mínima verificável. Os resultados em código real (2/2 e 3/3 em contratos de auditorias públicas, sem ajuste específico por contrato) indicam que esse resultado não depende de o contrato ter sido escrito para o benchmark.

A limitação principal, medida diretamente (Seção 5.3), é de cobertura: 63% dos achados de um contrato real de produção caem fora do catálogo formalizável atual, e a classe mais explorada em incidentes reais — reentrância — permanece fora do Certora por decisão deliberada, mantida como evidência de revisão estática até que a propriedade formal necessária (ordem entre chamada externa e escrita de estado, não apenas condição de valor) esteja validada; uma primeira versão dessa especificação (`specs/defivault_reentrancy_cei.spec`) foi escrita mas ainda não validada contra os dois lados do teste (deve falhar no contrato vulnerável, passar no corrigido).

Uma segunda limitação é de ambiente. Os projetos reais antes bloqueados foram em grande parte liberados (Seção 5.4), mas 63 de 264 arquivos ainda não são analisáveis: dependência privada indisponível (28 do Astaria), *stack too deep* (19 do Maia) e testes de `forge-std`. São limites do ambiente de teste, não da pipeline em si.

Por fim, sobre o objetivo de gás citado no título do plano de trabalho: o resultado deste período de pesquisa é a construção da salvaguarda de verificação formal em si — a capacidade de provar que uma mudança no contrato preserva as propriedades de segurança esperadas — que é o pré-requisito necessário para que otimizações de gás possam ser aplicadas com segurança. A condução de experimentos de otimização de gás sobre essa salvaguarda, usando o benchmark e os contratos reais já validados como base, é o passo natural seguinte e está listada em trabalhos futuros.

7. Conclusão

Este trabalho implementou e validou uma pipeline automatizada que integra análise estática, modelos de linguagem e verificação formal para identificar, corrigir e provar formalmente a correção de vulnerabilidades em contratos inteligentes Solidity. A validação em benchmark controlado (33/33 correções resolvidas) e em contratos reais de auditorias públicas (5/5 correções resolvidas em dois projetos Code4rena distintos, mais três controles negativos corretos) mostra que a abordagem generaliza para código não escrito especificamente para o projeto. A medição de

cobertura em contrato real (17% formalizável) expôs quantitativamente o limite da abordagem original — indexação manual por detector — e motivou a segunda frente do trabalho: prompts de perspectiva, contexto por grafo de chamadas e parsing independente de compilador, todos incorporados à pipeline e testados. Os passos seguintes mais imediatos são validar formalmente a propriedade de reentrância, desbloquear os projetos reais restantes com acesso de rede, e conduzir o experimento de otimização de gás propriamente dito sobre a base já verificada.

8. Referências

CHEN, Yizhou et al. From Cryptic to Clear: Training on LLM Explanations to Detect Smart Contract Vulnerabilities. **ACM Transactions on Software Engineering and Methodology**, [S. l.], 2025. DOI: 10.1145/3765753. Disponível em: https://dl.acm.org/doi/10.1145/3765753. Acesso em: 18 set. 2026.

NGUYEN, Nhat-Minh et al. Heterogeneous Graph Transformers with Large Language Models for Smart Contract Vulnerability Detection. **ACM Transactions on Software Engineering and Methodology**, [S. l.], 2025. DOI: 10.1145/3765751. Disponível em: https://dl.acm.org/doi/10.1145/3765751. Acesso em: 18 set. 2026.

LI, Ziwei et al. Enhancing Smart Contract Code–Comment Inconsistencies Detection via Two-Stage Pre-Training. **ACM Transactions on Software Engineering and Methodology**, [S. l.], 2025. DOI: 10.1145/3764867. Disponível em: https://dl.acm.org/doi/10.1145/3764867. Acesso em: 18 set. 2026.

FEIST, Josselin; GRIECO, Gustavo; GROCE, Alex. Slither: a static analysis framework for smart contracts. In: IEEE/ACM INTERNATIONAL WORKSHOP ON EMERGING TRENDS IN SOFTWARE ENGINEERING FOR BLOCKCHAIN (WETSEB), 2., 2019, Montreal. **Anais [...]**. Montreal: IEEE, 2019. p. 8-15.

CERTORA. **Certora Prover Documentation**. [S. l.]: Certora, 2026. Disponível em: https://docs.certora.com. Acesso em: 18 set. 2026.